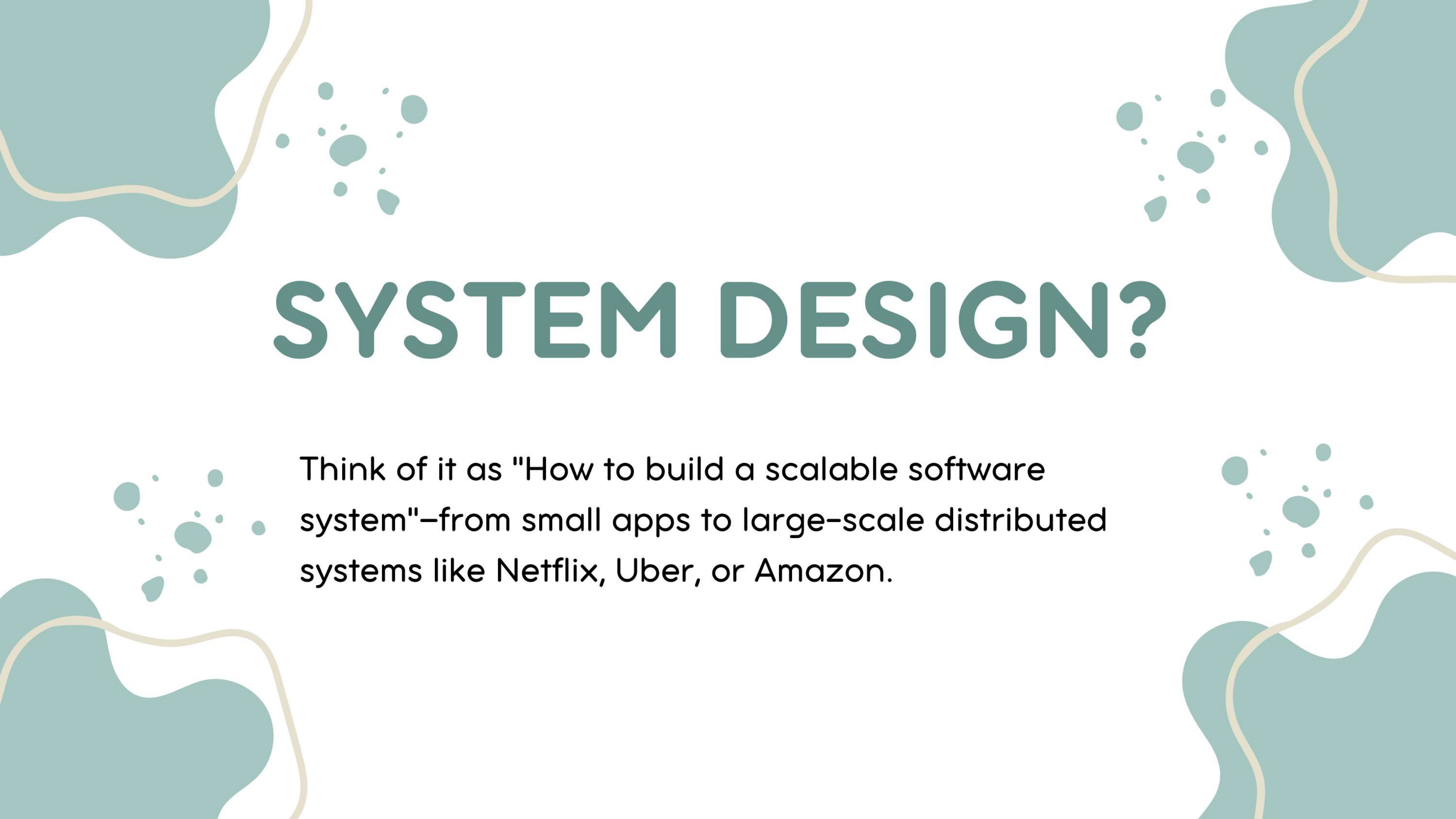




PRESENTATION

By Mohamed Ayamn, Alaa sherif



SYSTEM DESIGN?

Think of it as "How to build a scalable software system"—from small apps to large-scale distributed systems like Netflix, Uber, or Amazon.

SYSYEM DESIGN

System Design: is the process of defining the architecture, components, modules, interfaces, and data flow for a software system to meet specific requirements. It focuses on scalability, performance, reliability, maintainability, and security.

SYSTEM DESIGN

**involves creating both
a High-Level Design (HLD), which is like a roadmap
showing the overall plan,
and a Low-Level Design (LLD), which is a detailed
guide for programmers on how to build each part.
It ensures a well-organized and smoothly functioning
project. High-Level Design and Low-Level Design are
the two main aspects of System Design.**

SYSTEM DESIGN

High Level Design vs Low Level Design

HLD High Level Design



Structure,
components
& relationships



Architects

VS

LLD Low Level Design



Implementation
details



Engineers

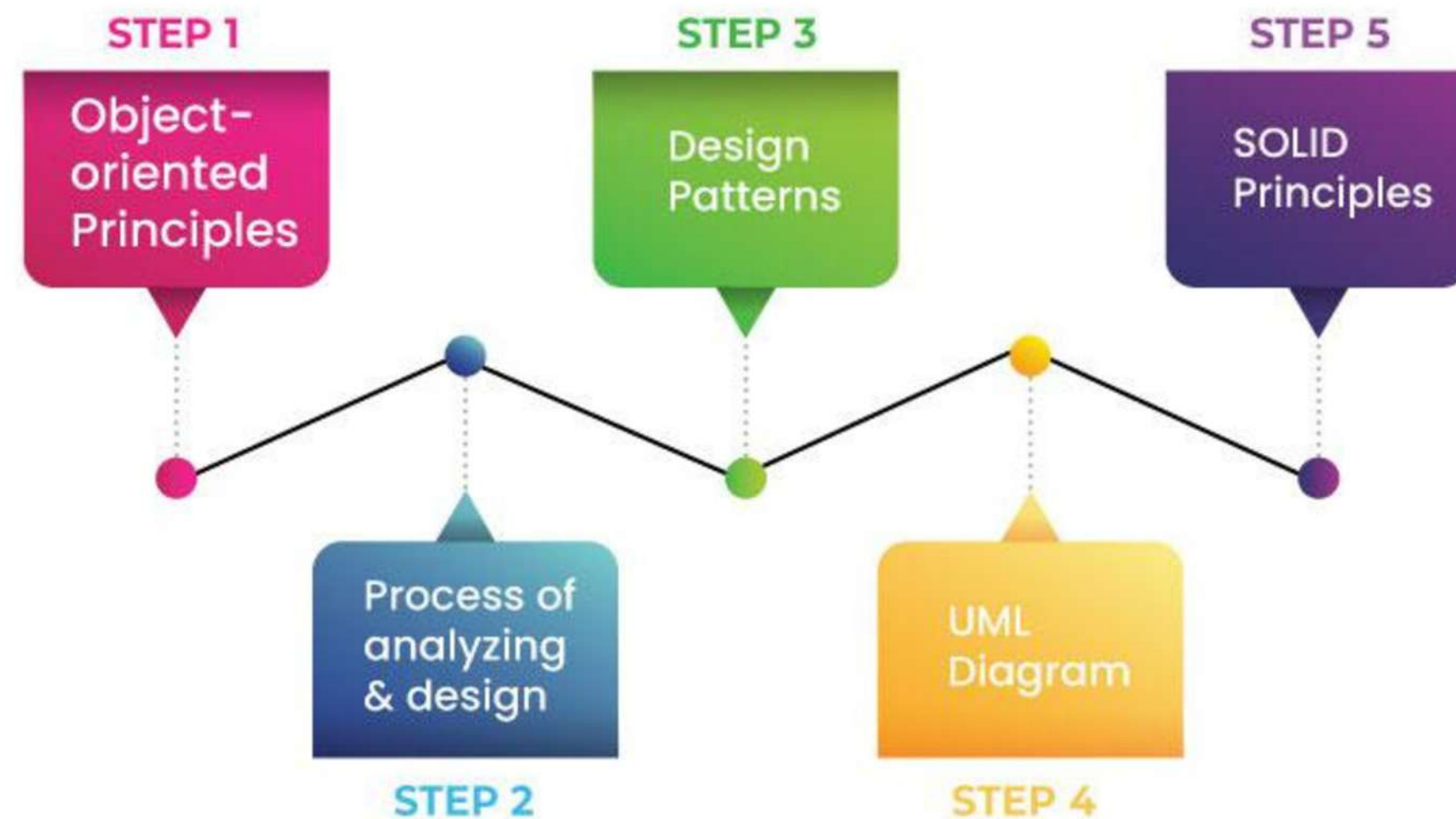
Describes

Typically
created by

Difference between High Level Design and Low Level Design

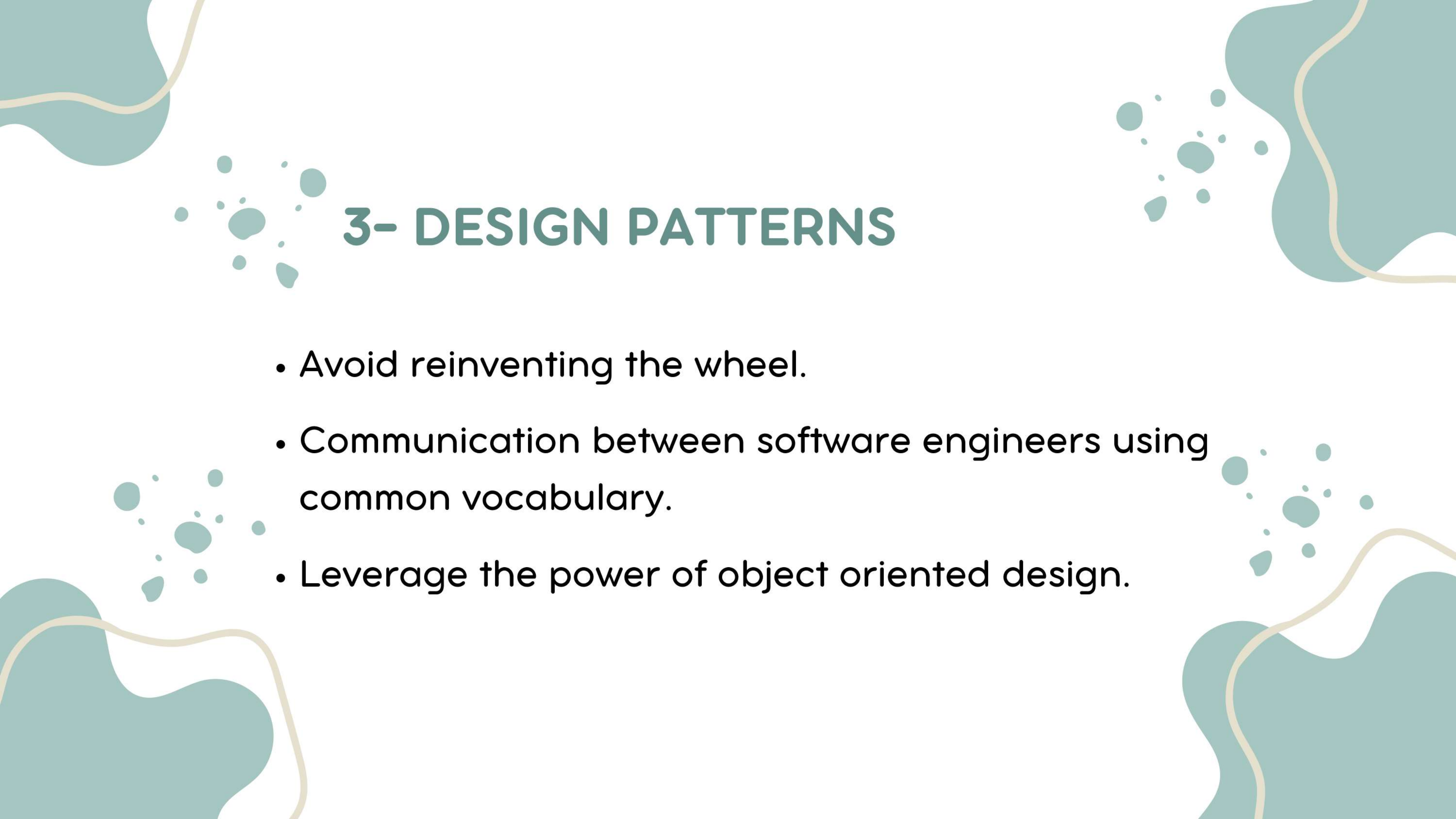


ROADMAP TO FORM LLD FROM HLD?



1- OOP (OBJECT-ORIENTED PRINCIPLES)

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction



3- DESIGN PATTERNS

- Avoid reinventing the wheel.
- Communication between software engineers using common vocabulary.
- Leverage the power of object oriented design.

3.1-WHAT DOES THE PATTERN CONSIST OF?

- Name
- Intent
- Motivation
- Structure
- Code Examples

3.2- DESIGN PATTERNS TYPES

be **PATTERNS**

Design
Pattern

Creational
Design Pattern

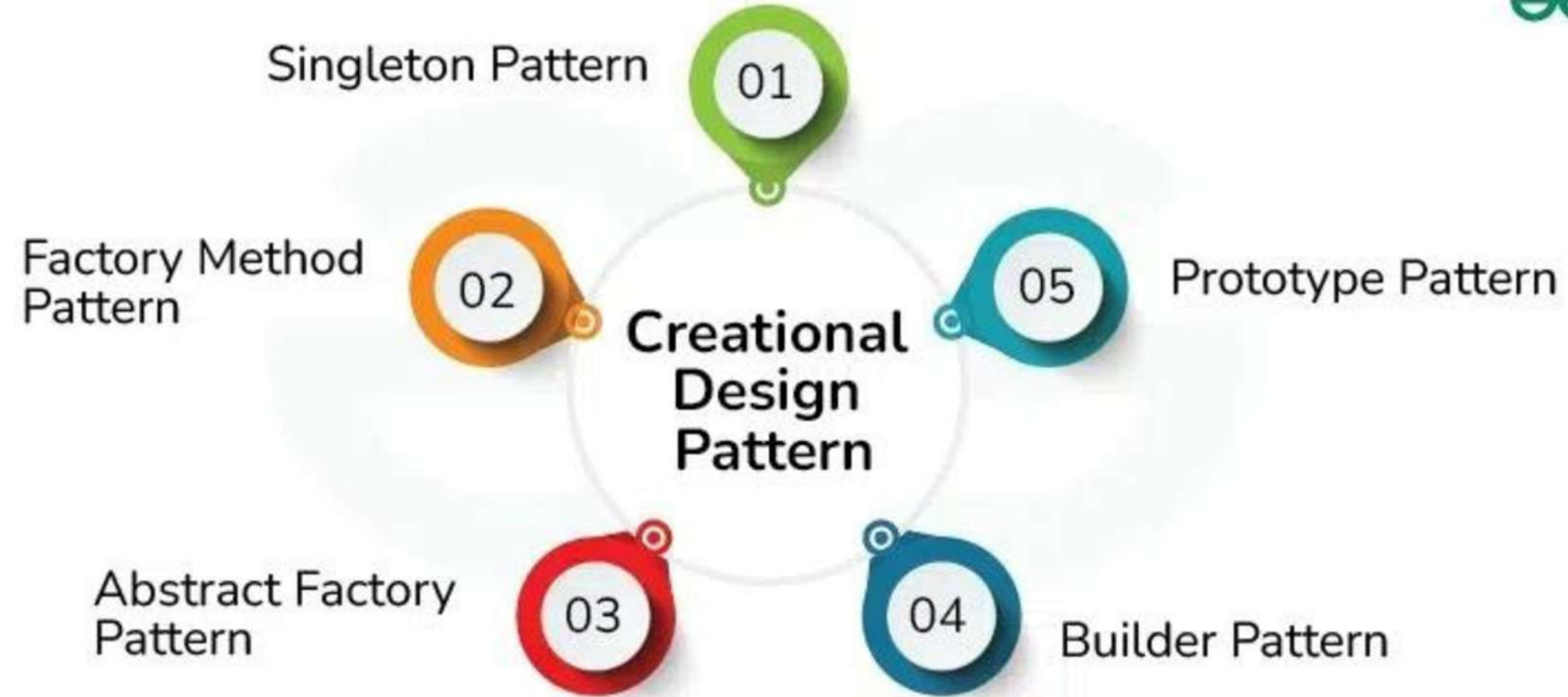
Structural
Design Pattern

Behavioral
Design Pattern



bepatterns.com

3.2.1 CREATIONAL



3.2.1 CREATIONAL

1- Factory Method

- Name: Factory Method Pattern

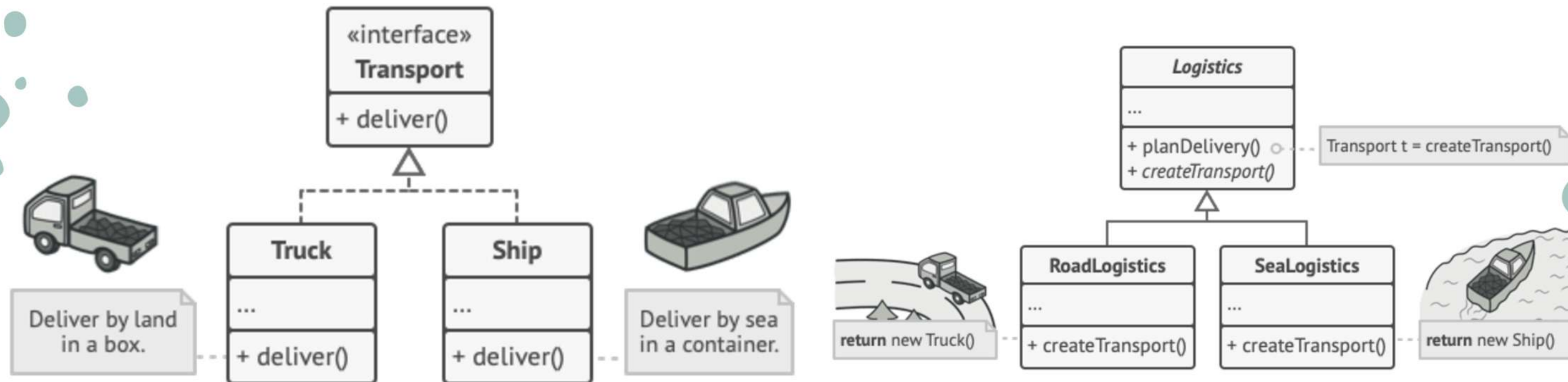
- Intent:

- Define an interface for creating an object, but let sub classes decide which class to instantiate.

3.2.1 CREATIONAL

1- Factory Method Pattern

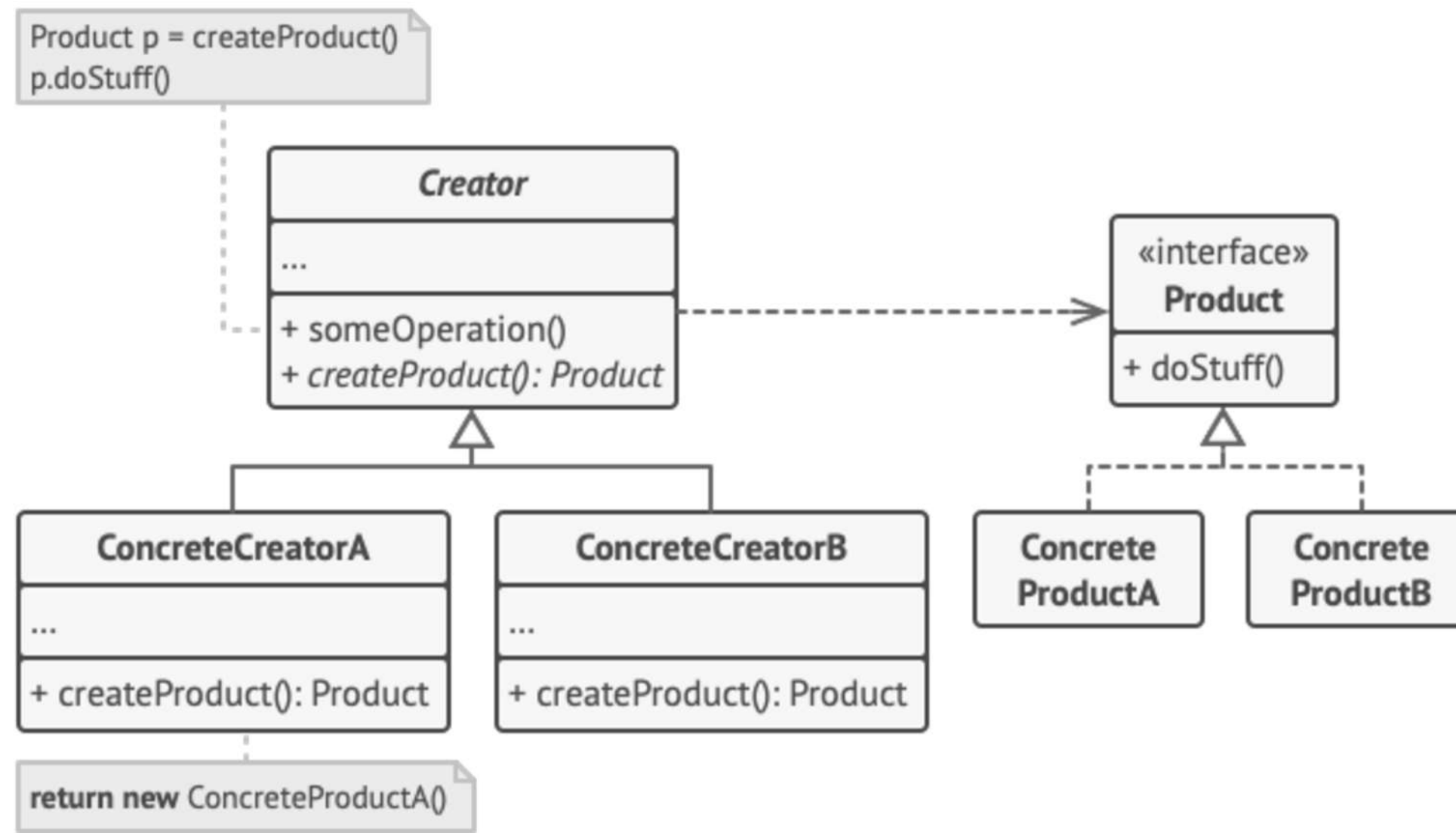
- Motivation:



3.2.2 CREATIONAL

1- Factory Method Pattern

- Structure



3.2.2 CREATIONAL

1- Factory Method Pattern

- Code example

3.2.1 CREATIONAL

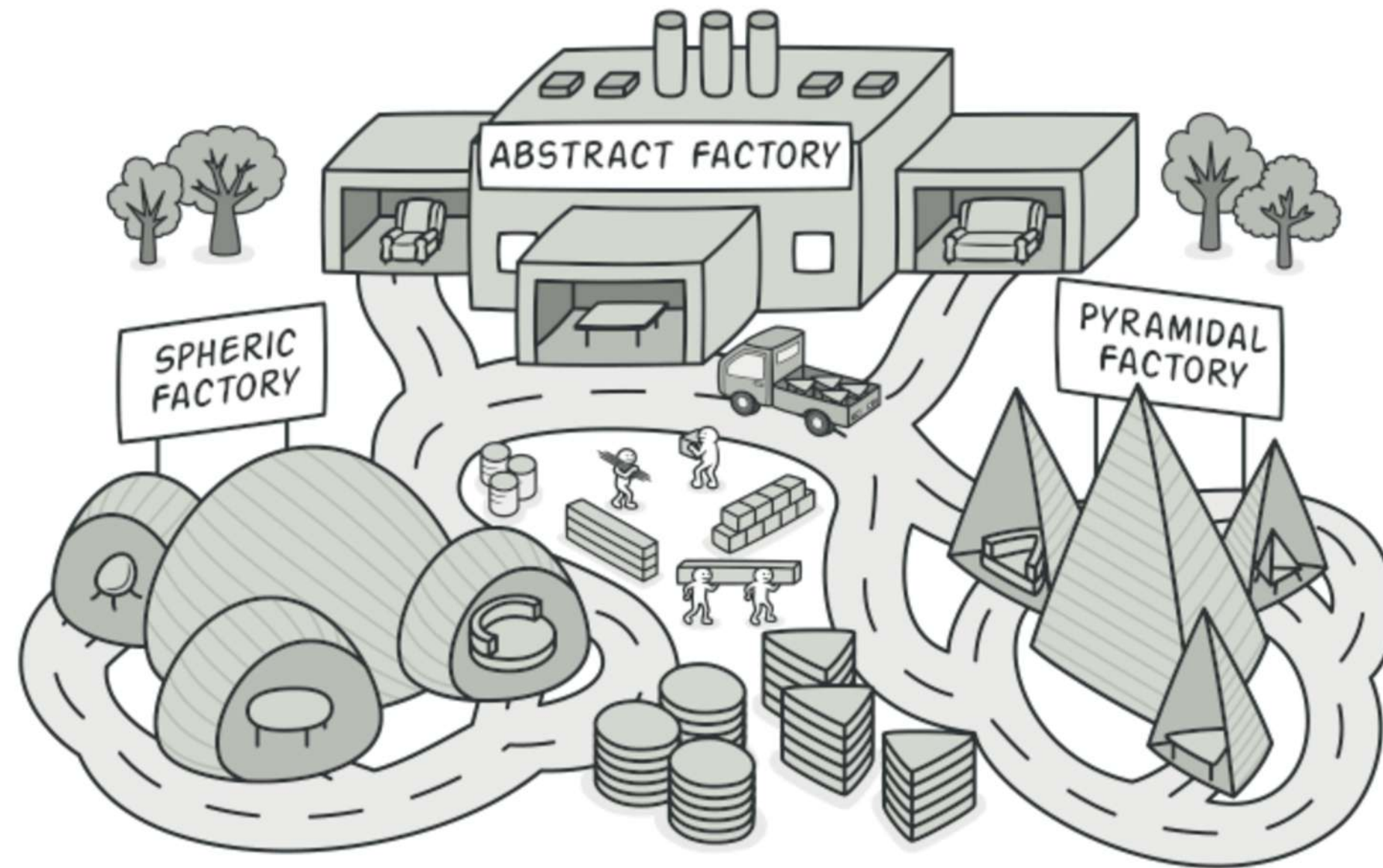
2- Abstract Factory

- Name: Abstract Factory Pattern
- Intent:
- Abstract Factory is a creational design pattern that lets you produce **families** of related objects without specifying their concrete classes.

3.2.1 CREATIONAL

2- Abstract Factory Pattern

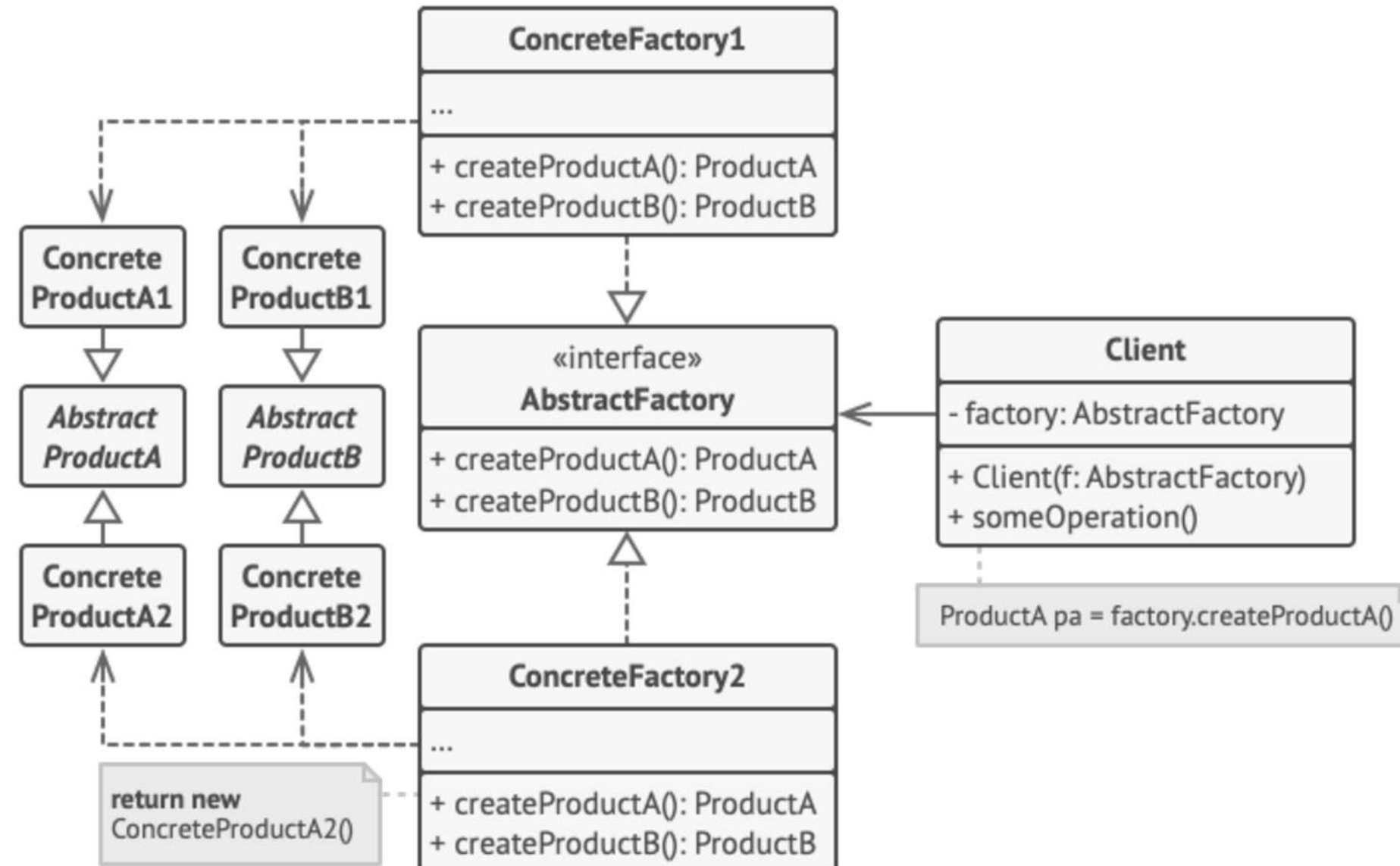
- Motivation:



3.2.2 CREATIONAL

2- Abstract Factory Pattern

- Structure



3.2.2 CREATIONAL

2- Abstract Factory Pattern

- Code example

3.2.1 CREATIONAL

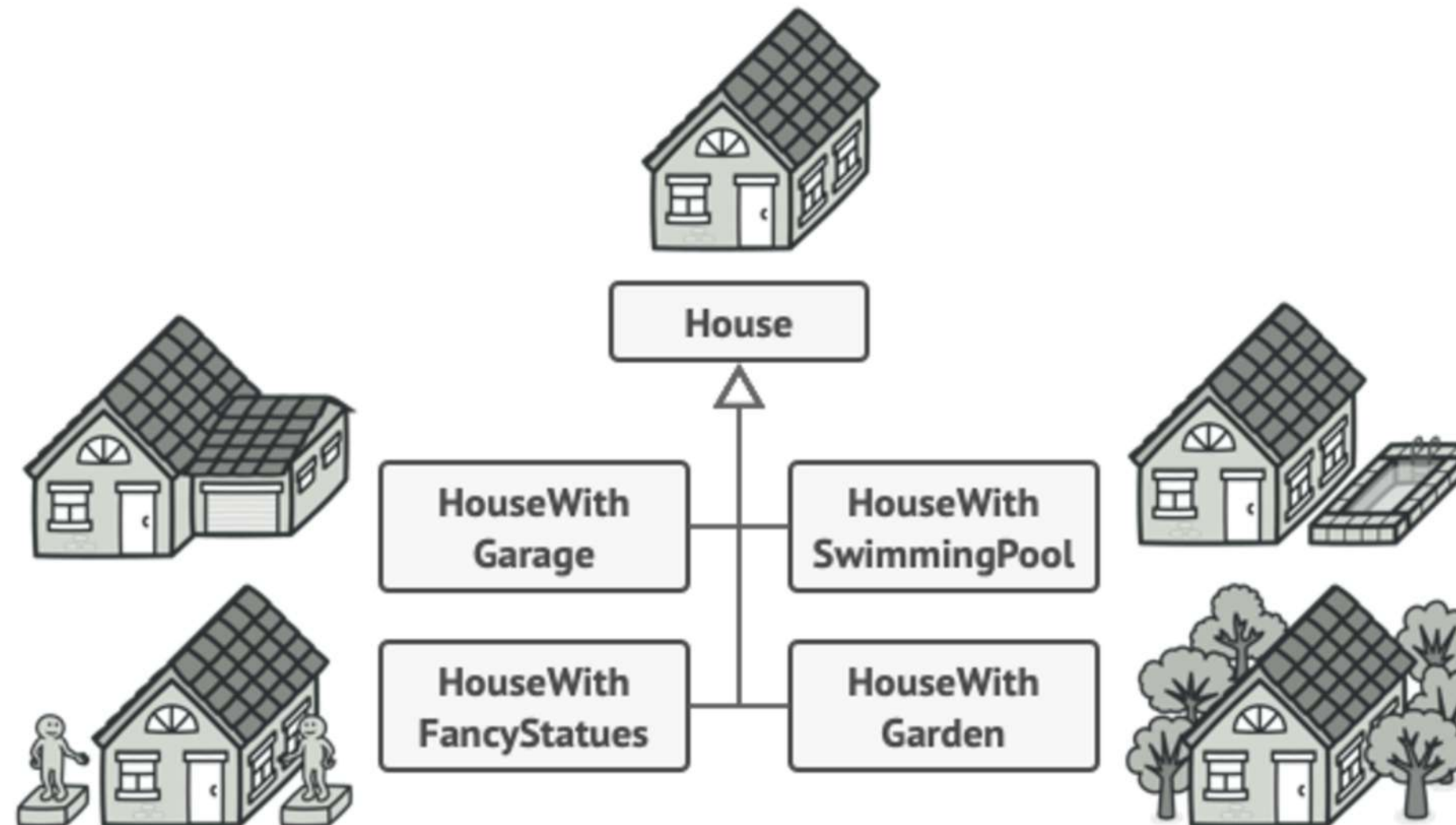
3- Builder

- Name: Builder Pattern
- Intent:
- Builder is a creational design pattern that lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code.

3.2.1 CREATIONAL

3- Builder Pattern

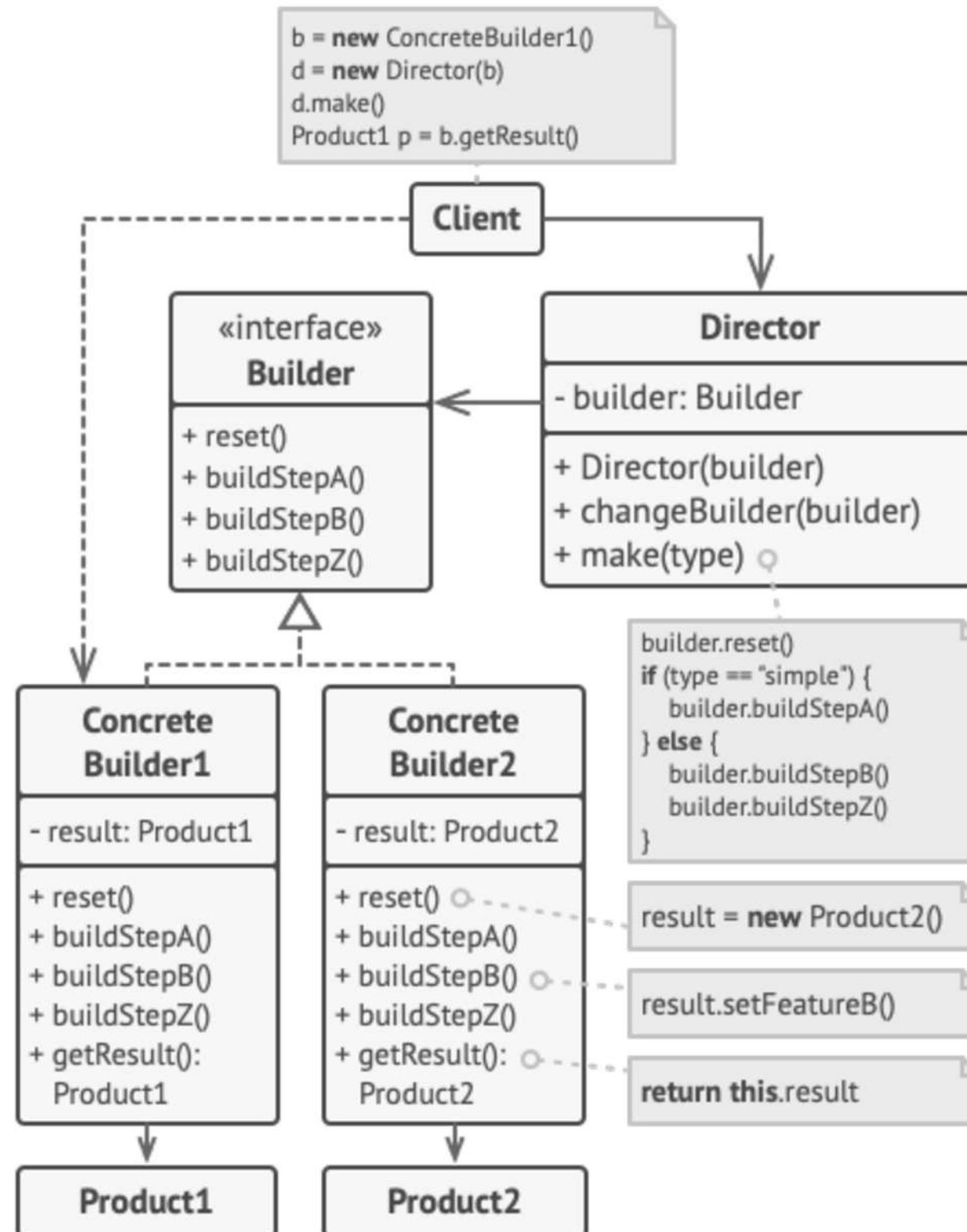
- Motivation:



3.2.2 CREATIONAL

3- Builder Pattern

- Structure



3.2.2 CREATIONAL

3- Builder Pattern

- Code example

3.2.1 CREATIONAL

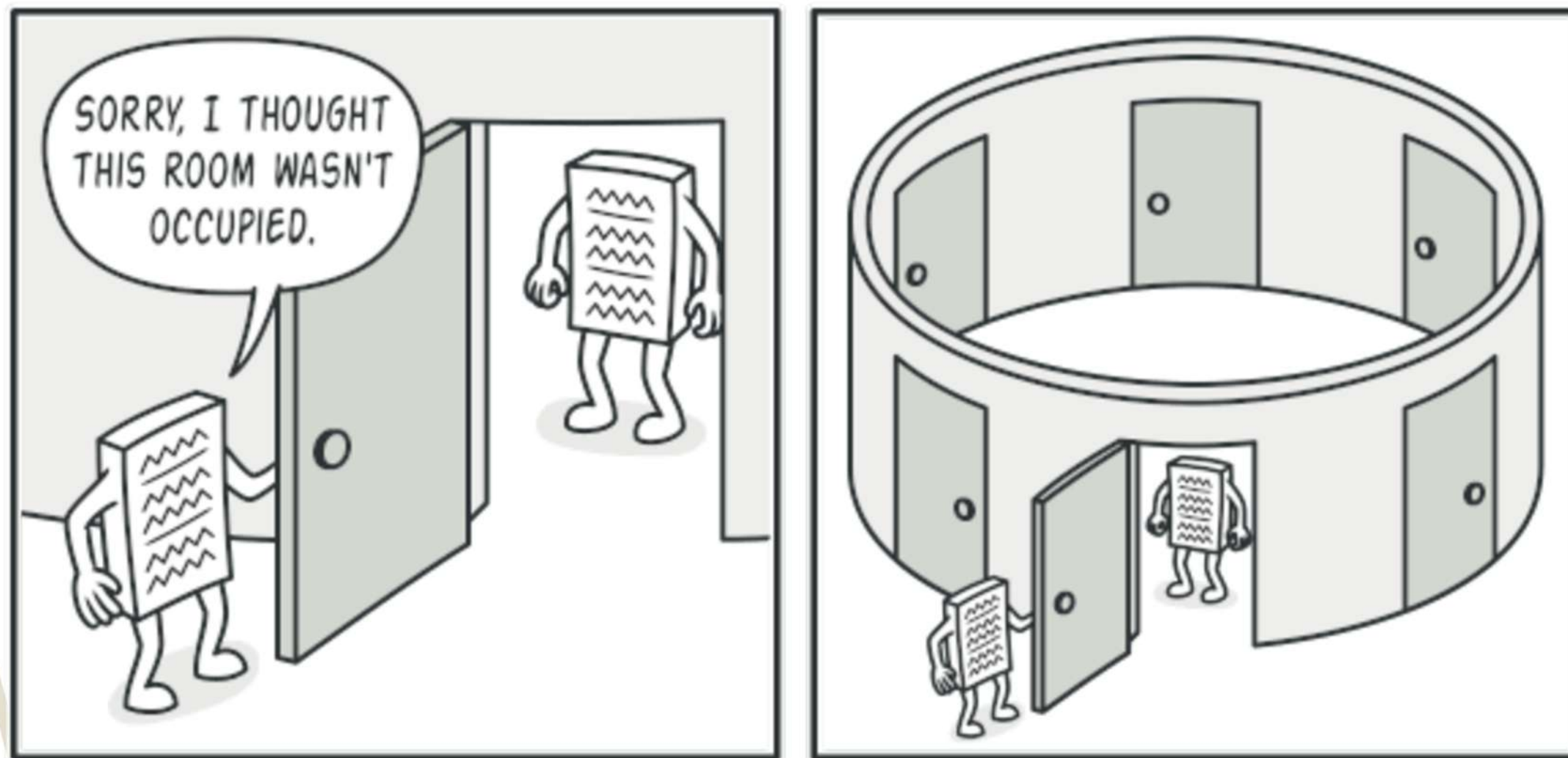
4- Singleton pattern

- Name: Singleton Pattern
- Intent:
- Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.

3.2.1 CREATIONAL

4- Singleton pattern

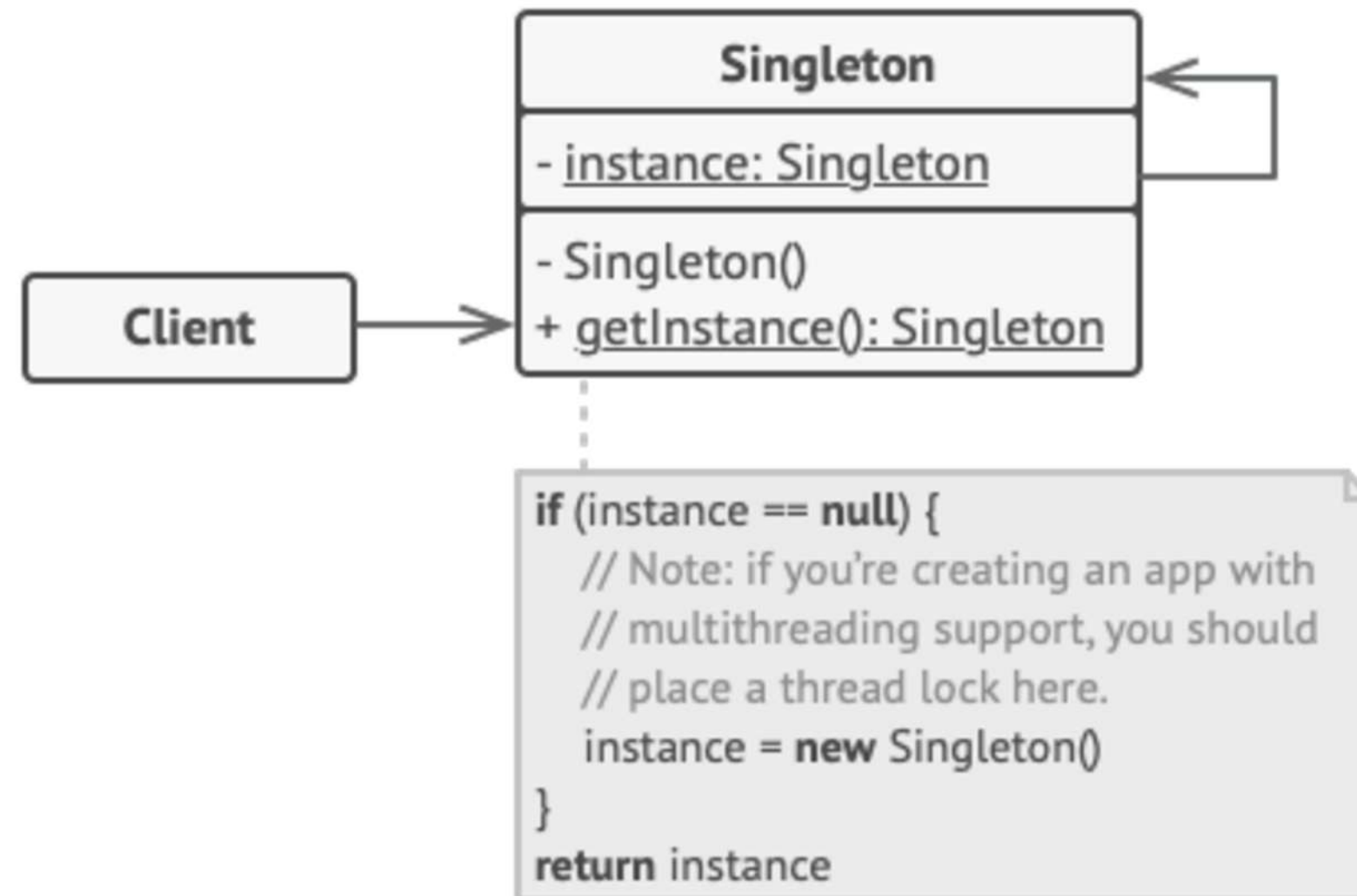
- Motivation:



3.2.2 CREATIONAL

4- Singleton pattern

- Structure

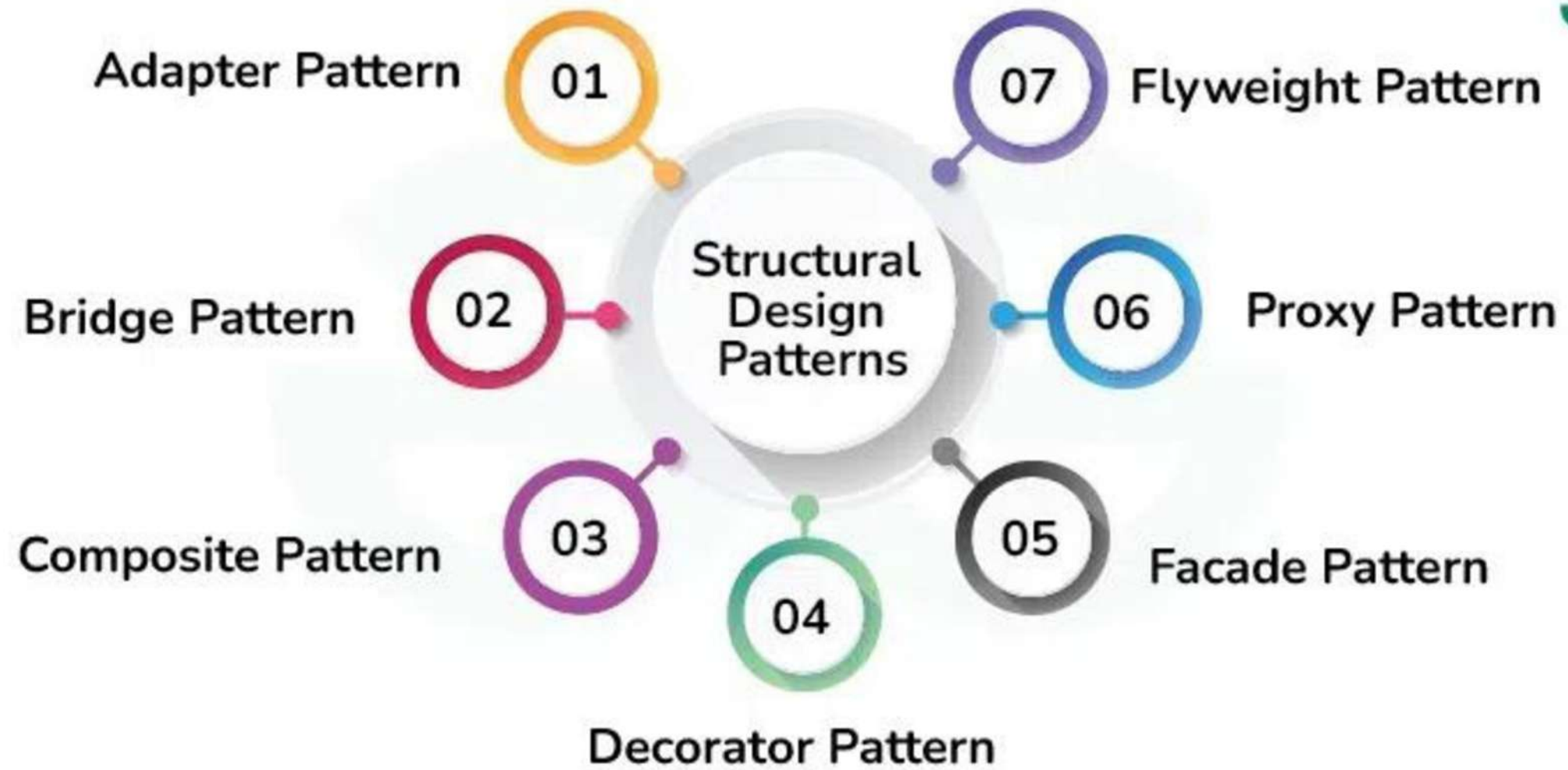


3.2.2 CREATIONAL

4- Singleton pattern

- Code example

3.2.2 STRUCTURAL



3.2.2 STRUCTURAL

1- Adapter Pattern

- Name: Adapter Pattern
- Intent:
 - The Adapter Pattern allows two incompatible interfaces to work together by providing a wrapper that translates one interface into another.
 - It acts as a bridge between different systems or components.

3.2.2 STRUCTURAL

1- Adapter Pattern

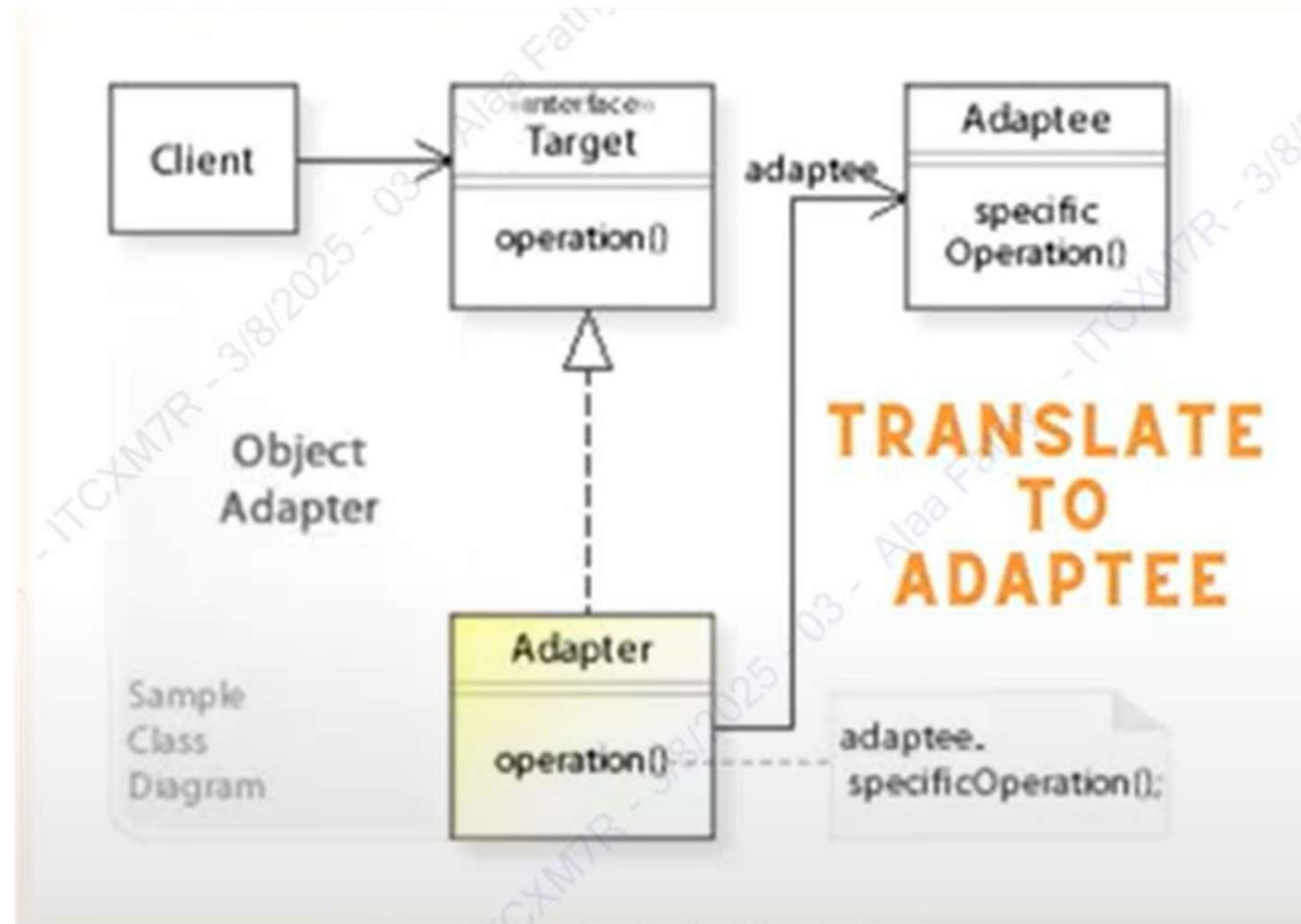
- Motivation:



3.2.2 STRUCTURAL

1- Adapter Pattern

- Structure



3.2.2 STRUCTURAL

1- Adapter Pattern

- Code example

3.2.2 STRUCTURAL

2- Proxy Pattern

- Name: Proxy Pattern
- Intent:
 - Provide a placeholder for another object to control access to it.
 - Can be used for lazy initialization, security, caching, logging, etc.

3.2.2 STRUCTURAL

2- Proxy Pattern

- Motivation:



client



Secratry

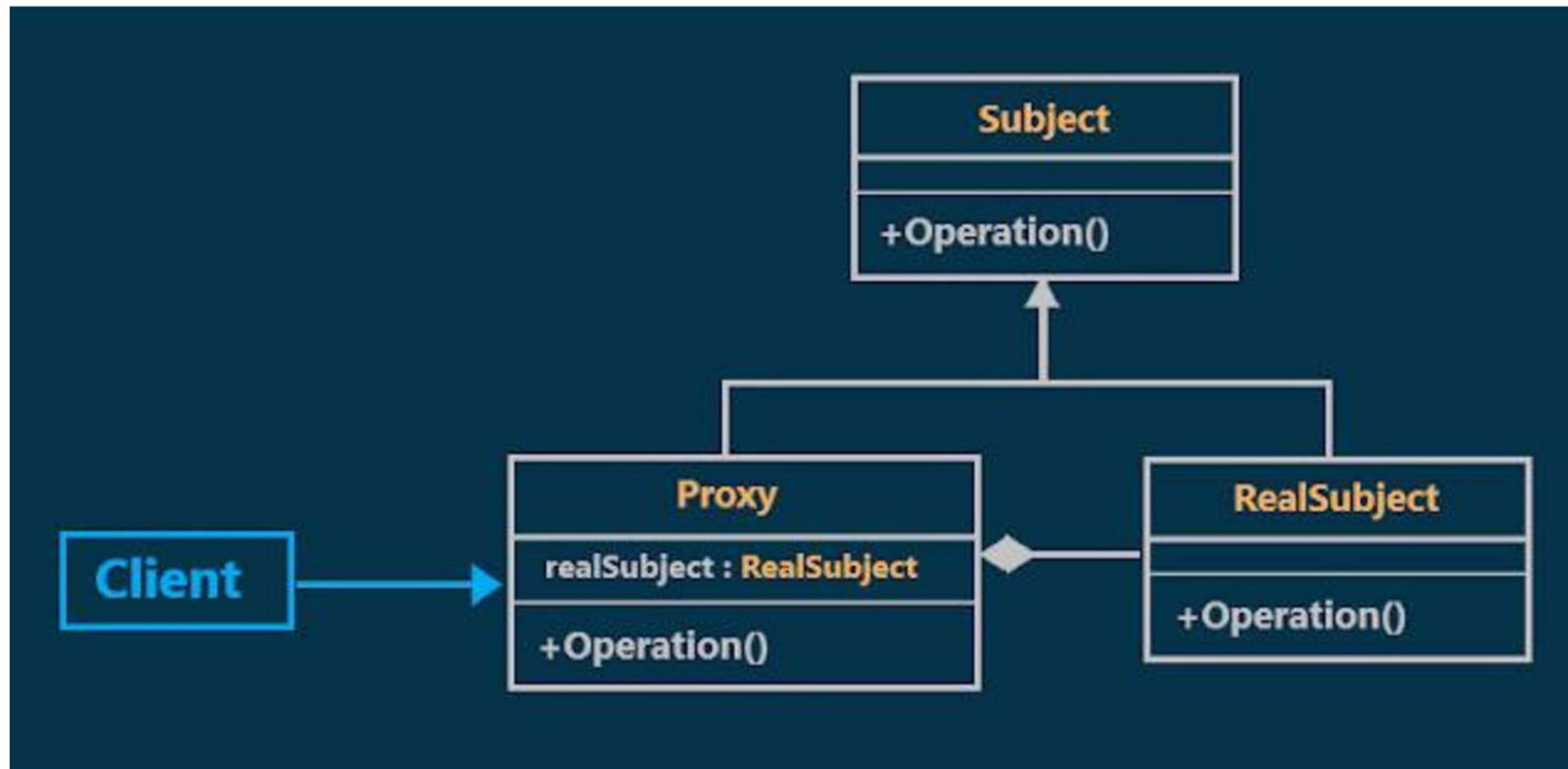


Manager

3.2.2 STRUCTURAL

2- Proxy Pattern

- Structure



3.2.2 STRUCTURAL

2- Proxy Pattern

- Code example

3.2.2 STRUCTURAL

3- Decorator Pattern

- Name: Decorator Pattern
- Intent:
 - Extend behavior dynamically without modifying existing classes.
 - Wrap the original object inside a decorator that provides extra functionality.

3.2.2 STRUCTURAL

3- Decorator Pattern

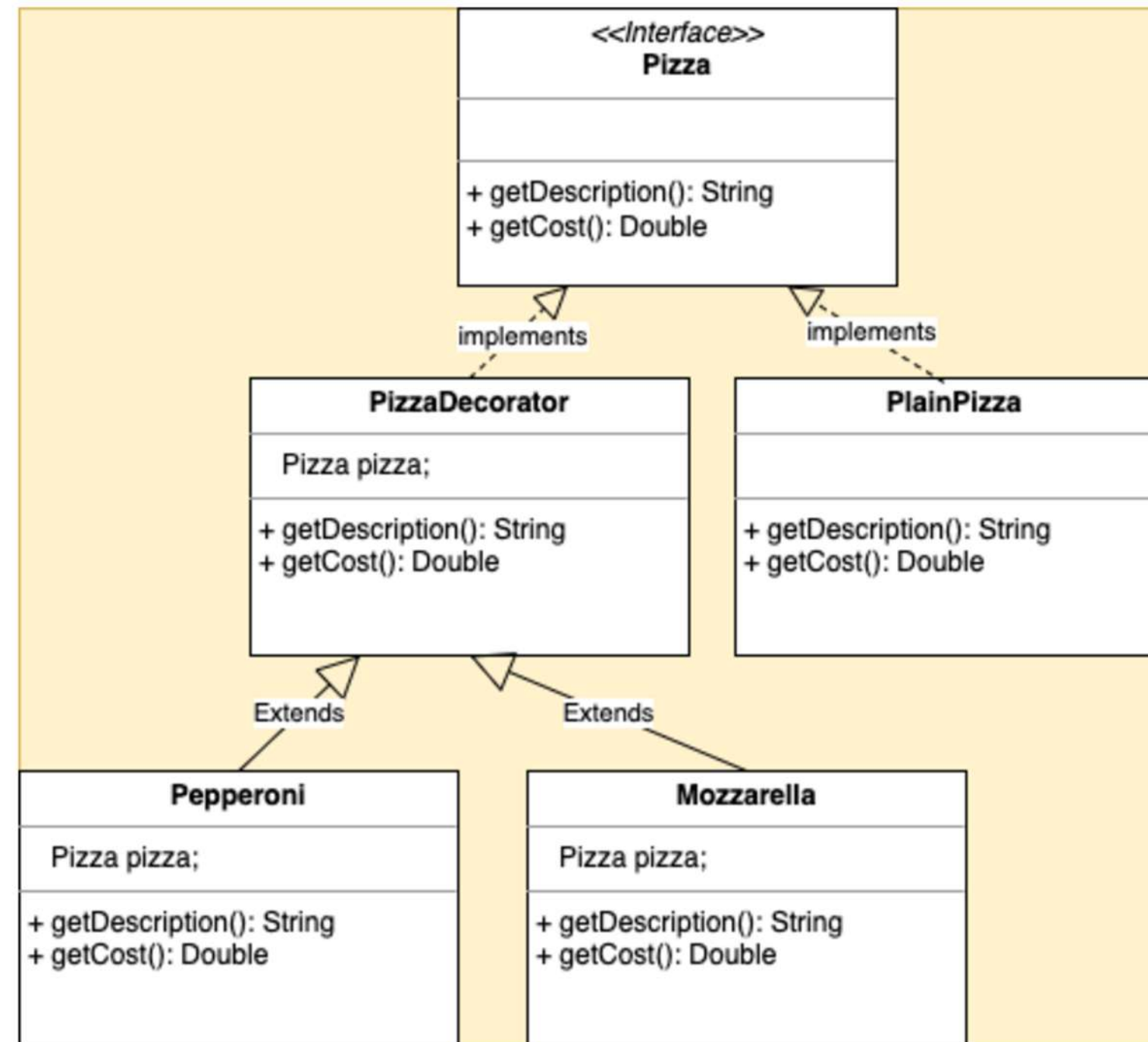
- Motivation:



3.2.2 STRUCTURAL

3- Decorator Pattern

- Structure



3.2.2 STRUCTURAL

3- Decorator Pattern

- Code example

3.2.2 STRUCTURAL

4- Facade Pattern

- Name: Facade Pattern
- Intent:
 - Provide a simplified interface to a complex system.
 - Encapsulate multiple subsystems behind a single unified interface.

3.2.2 STRUCTURAL

4- Facade Pattern

- Motivation:



I want to eat

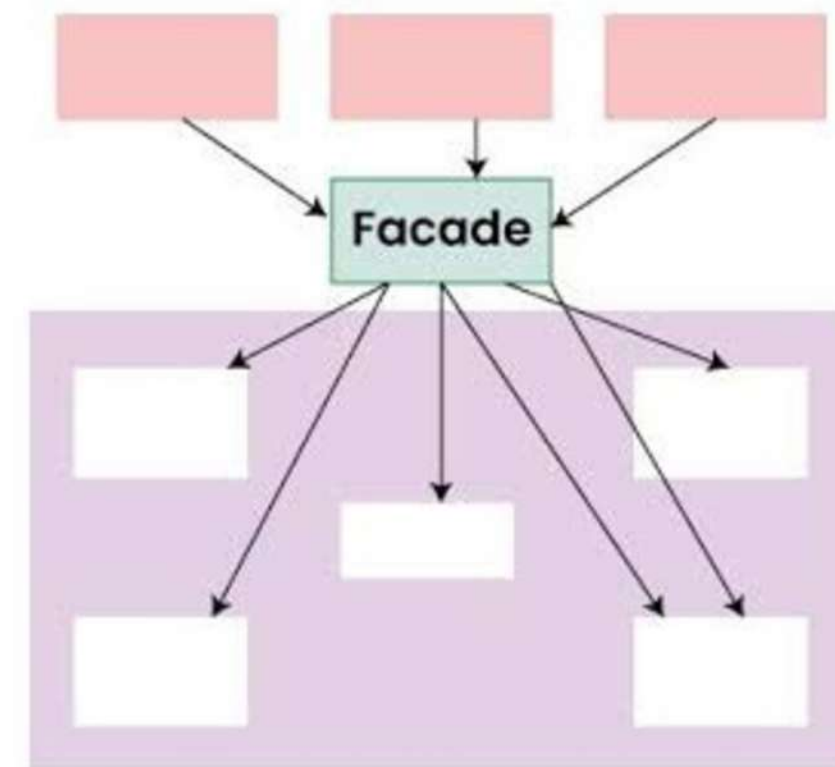
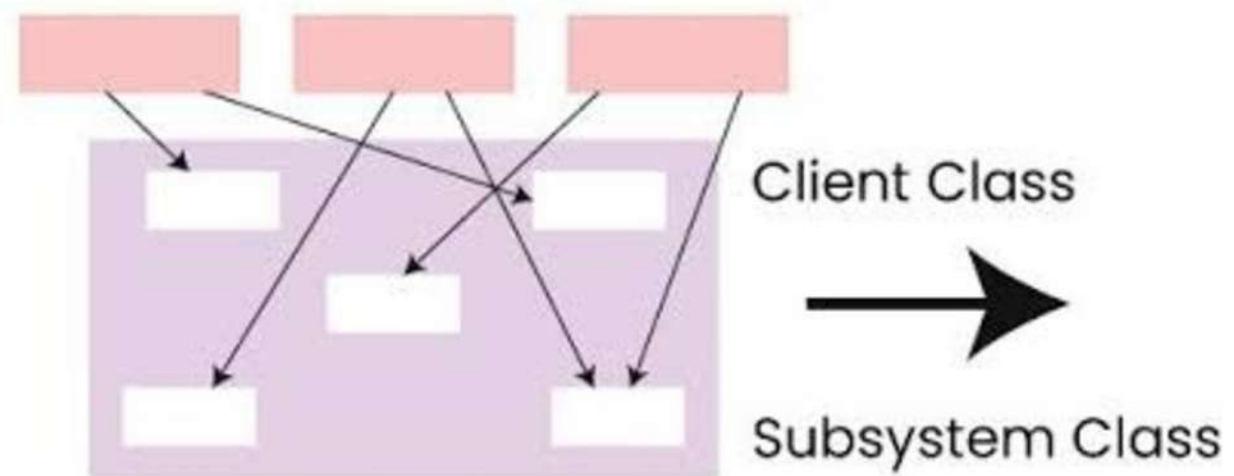


3.2.2 STRUCTURAL

4- Facade Pattern

- Structure

What is Facade Method Design Pattern



3.2.2 STRUCTURAL

4- Facade Pattern

- Code example

3.2.3 BEHAVIORAL



3.2.3 BEHAVIORAL

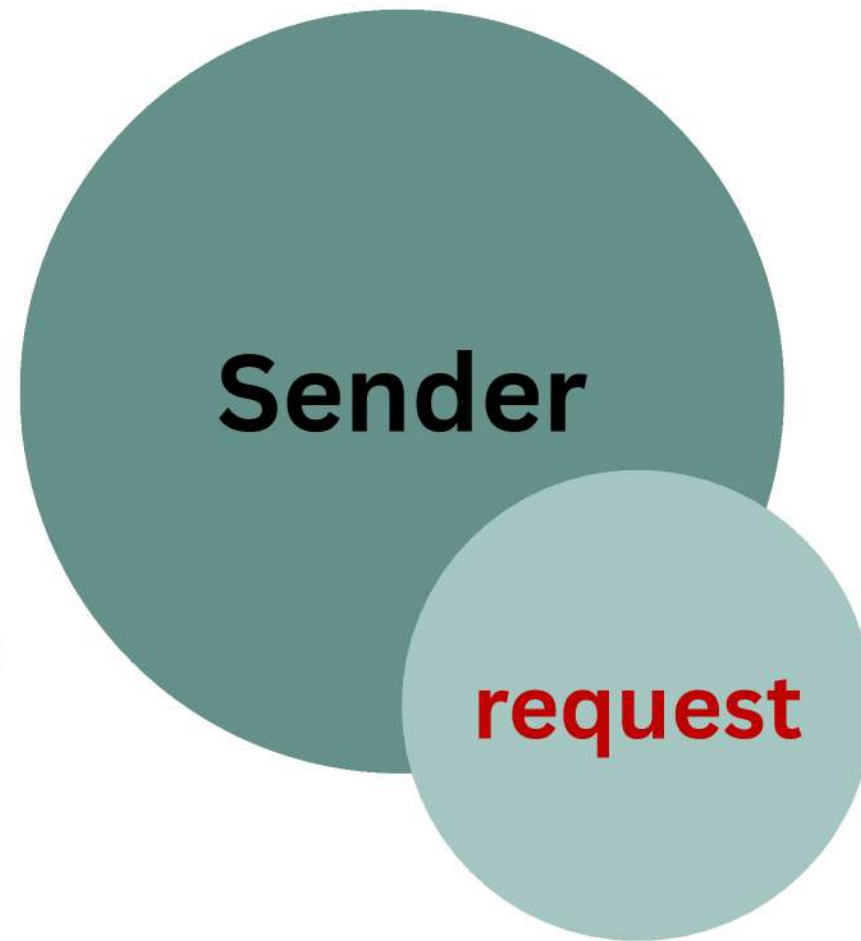
1- Chain Of Responsibility

- Name: Chain of Responsibility (CoR)
- Intent:
 - The Chain of Responsibility pattern is used to pass requests along a chain of handlers.
 - Each handler either processes the request or passes it to the next handler in the chain.
 - This decouples senders and receivers of requests.

3.2.3 BEHAVIORAL

1- Chain Of Responsibility

- Motivation:



- Can approve anything



- Can approve budget < 1500

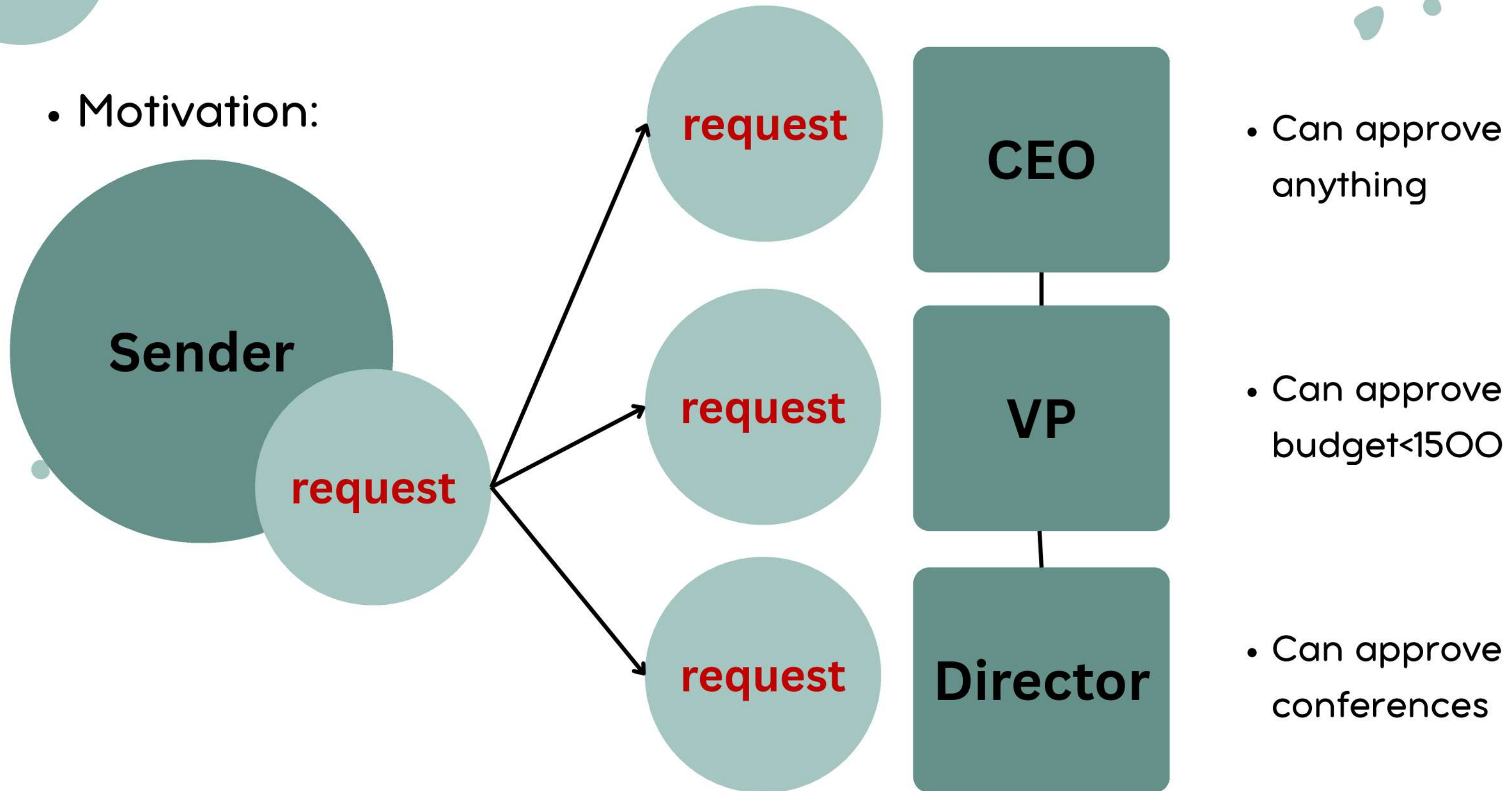


- Can approve conferences

3.2.3 BEHAVIORAL

1- Chain Of Responsibility

- Motivation:



3.2.3 BEHAVIORAL

1- Chain Of Responsibility

- Motivation:

Sender

request

CEO

- Can approve anything

VP

- Can approve budget < 1500

request

Director

- Can approve conferences

3.2.3 BEHAVIORAL

1- Chain Of Responsibility

- Motivation:

Sender

request

request

CEO

VP

Director

- Can approve anything

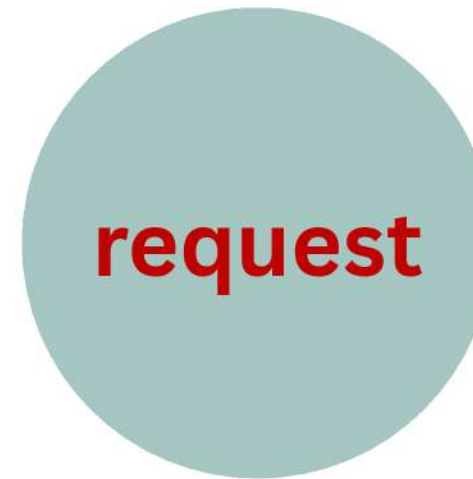
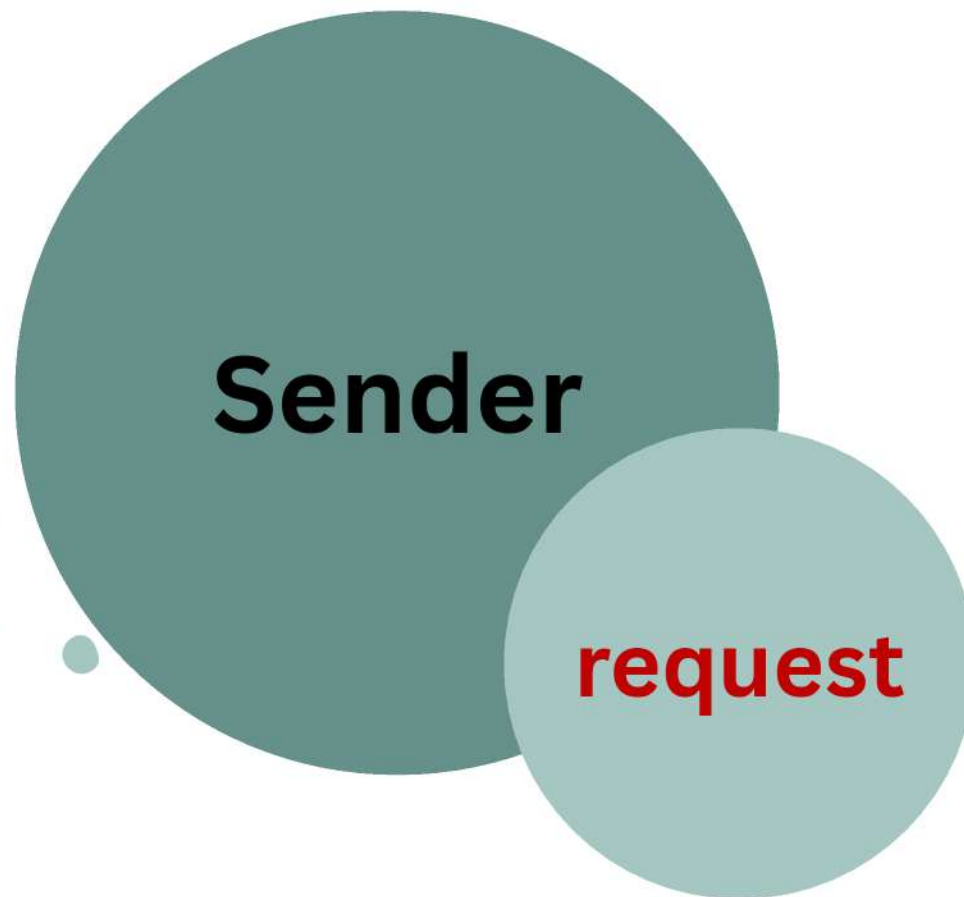
- Can approve budget<1500

- Can approve conferences

3.2.3 BEHAVIORAL

1- Chain Of Responsibility

- Motivation:



- Can approve anything



- Can approve budget<1500

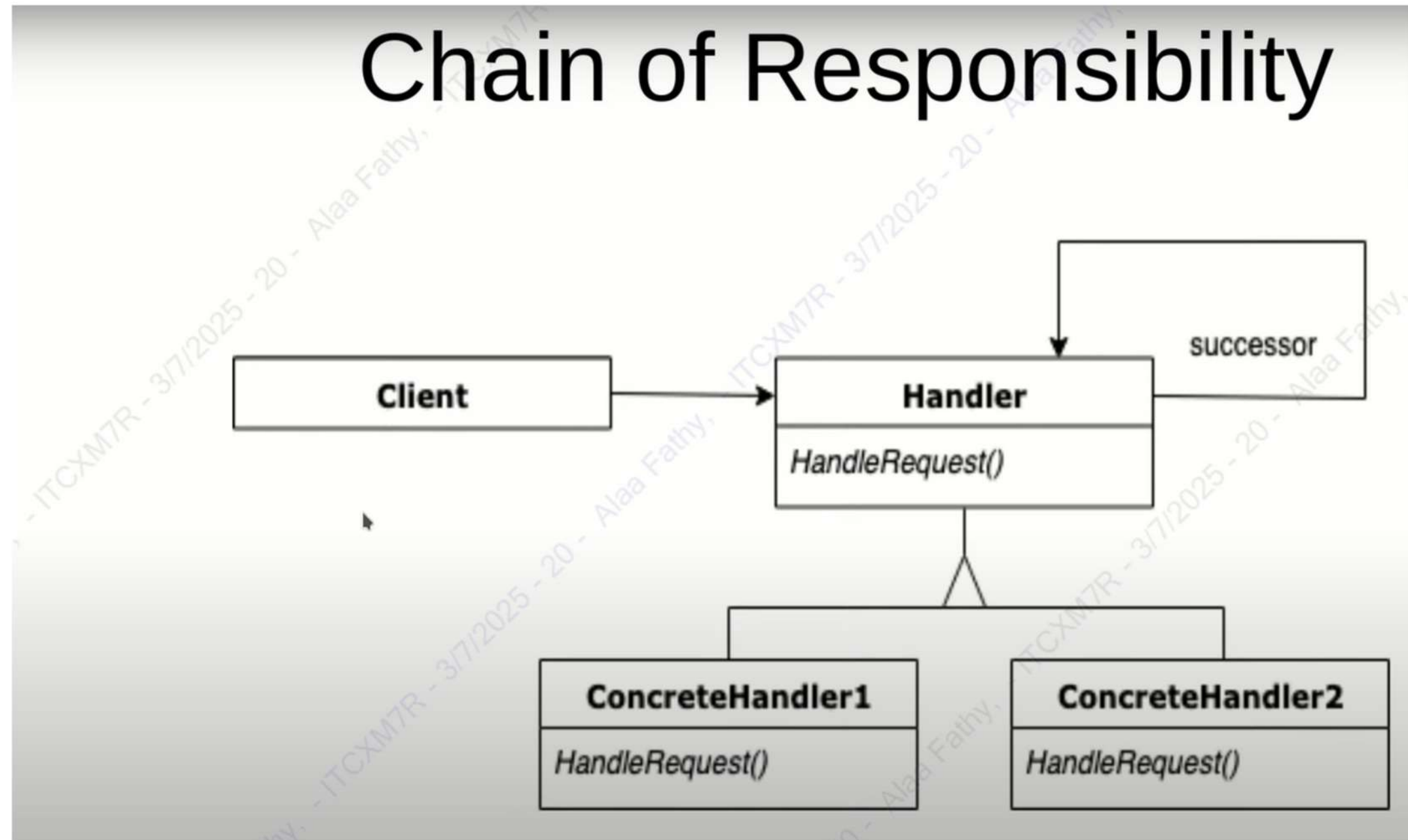


- Can approve conferences

3.2.3 BEHAVIORAL

1- Chain Of Responsibility

- Structure:



3.2.3 BEHAVIORAL

1- Chain Of Responsibility

- Code example

3.2.3 BEHAVIORAL

2- Observer

- Name: Observer
- Intent:
 - Defines a one-to-many dependency between objects.
 - When one object (Subject) changes state, all its dependents (Observers) are notified automatically.

3.2.3 BEHAVIORAL

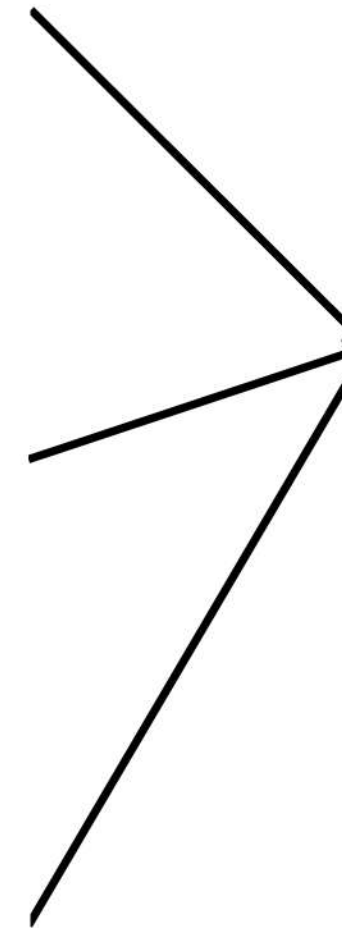
2- Observer

- Motivation:

Subject
(Observable)



Observers



3.2.3 BEHAVIORAL

2- Observer

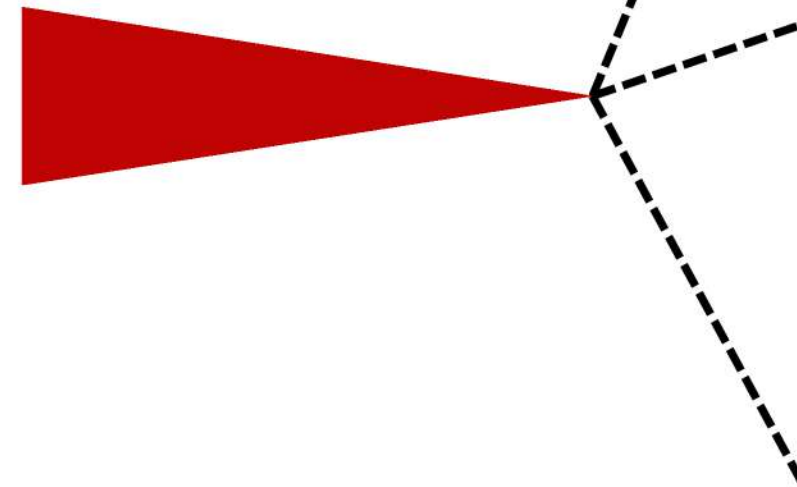
- Motivation:

Subject
(Observable)



When new video
upload

notifiy

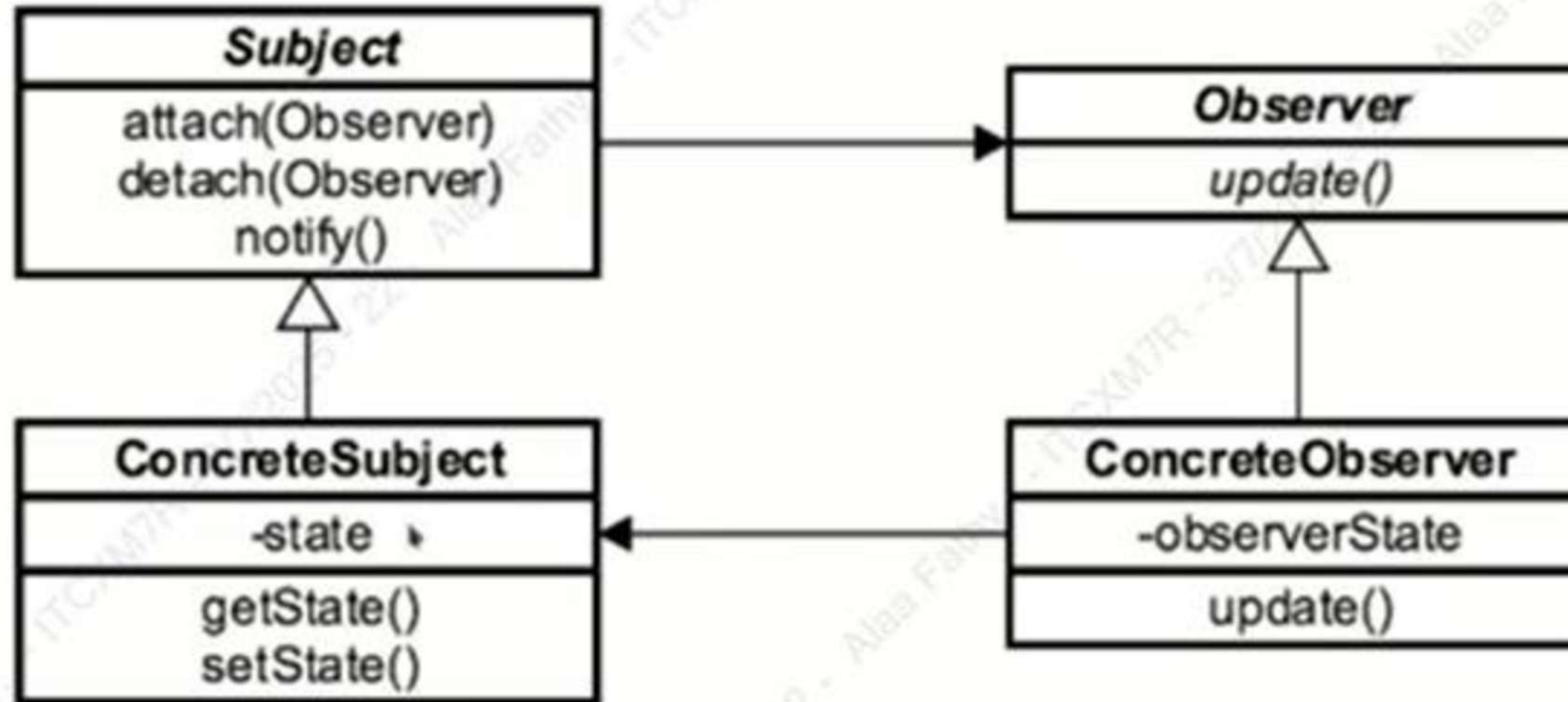


Observers

3.2.3 BEHAVIORAL

2- Observer

- Structure



3.2.3 BEHAVIORAL

2- Observer

- Code example

EXERCISE

- A Library Management System (LMS) is a software solution used to manage books, users, borrowing, and returning activities efficiently. The system ensures that books are available for borrowing, users can request books, and the library keeps track of stock and reservations.

EXERCISE

characteristic

✓ Requirements:

1. Create a only one LibraryService to manage books.
2. Seperate the logic of creation different types of book outside the entrie code.
3. We want to create A PremiumBook can allow extra 10 days for borrowing without modify the book entity.
4. Make premium access for EBooks.
5. Book borrowing requests go through Librarian → Manager.
6. Simplify the library operations handling Book Borrowing Requests.

Apply possible design patterns to solve each requirement.

EXERCISE

characteristic

✓ Requirements:

7. We can use external book data that is with format json and want to convert it to our book entity.

```
{  
  "bookTitle": "Design Patterns",  
  "isBorrowable": true,  
  "authorName": "Erich Gamma",  
  "year": 1994  
}
```

8. We want to notify users when books are available.

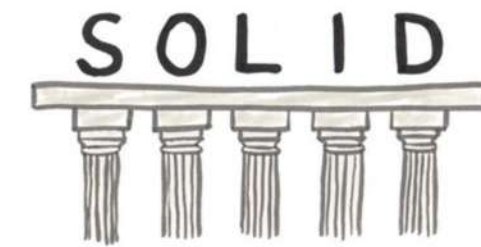
- **BONUS:**

9. Borrowing requests must be approved based on the number of days requested

- Librarian--->7 days
- Manager--->14 days
- Director---->15+ days

Apply possible design patterns to solve each requirement.

5-SOLID PRINCIPLES



SINGLE
RESPONSIBILITY



LISKOV
SUBSTITUTION



OPEN-CLOSED
PRINCIPLE



INTERFACE
SEGREGATION

DEPENDENCY
INVERSION



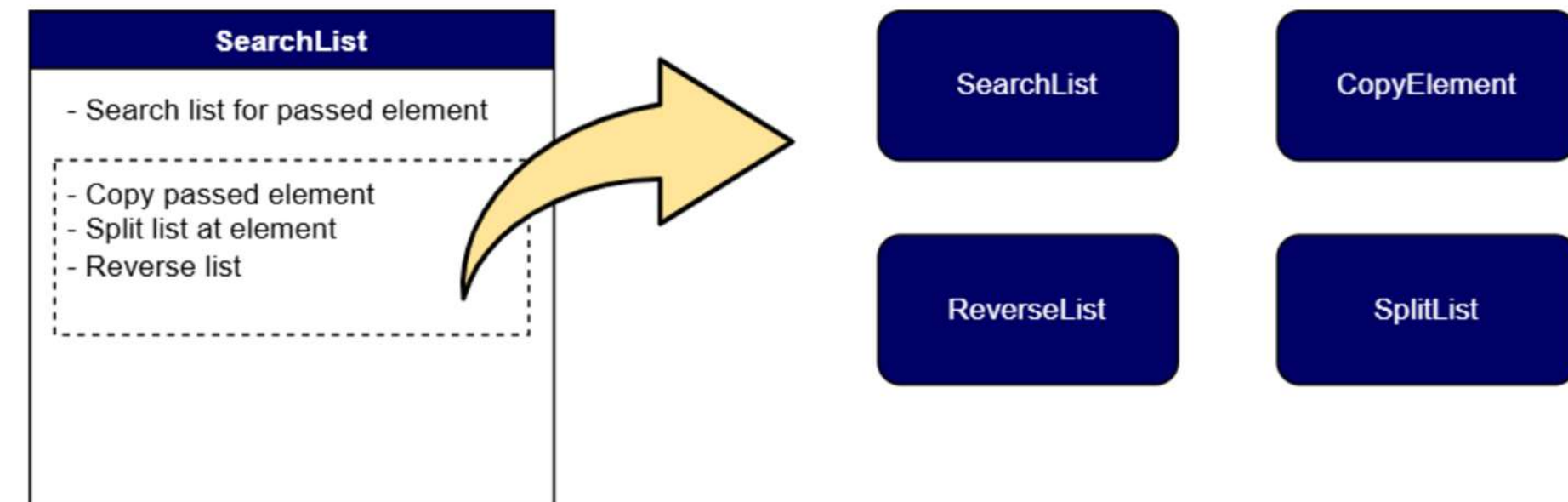
O5.1- **S**INGLE RESPONSIBILITY PRINCIPLE (SRP)

- A class should have only one reason to change.

- Each class should have one responsibility, one single purpose, This means that a class do only one job, which leads us to conclude

- it should have **only reason to change**.

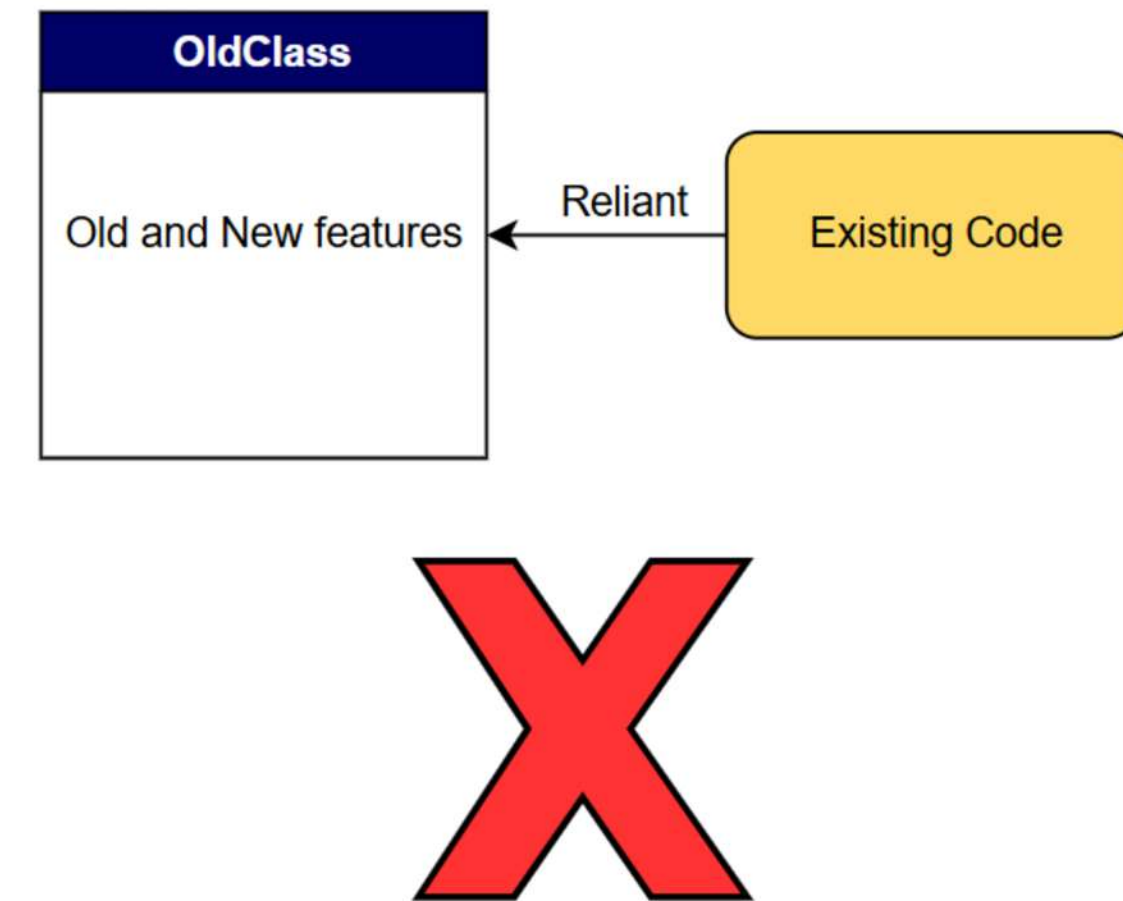
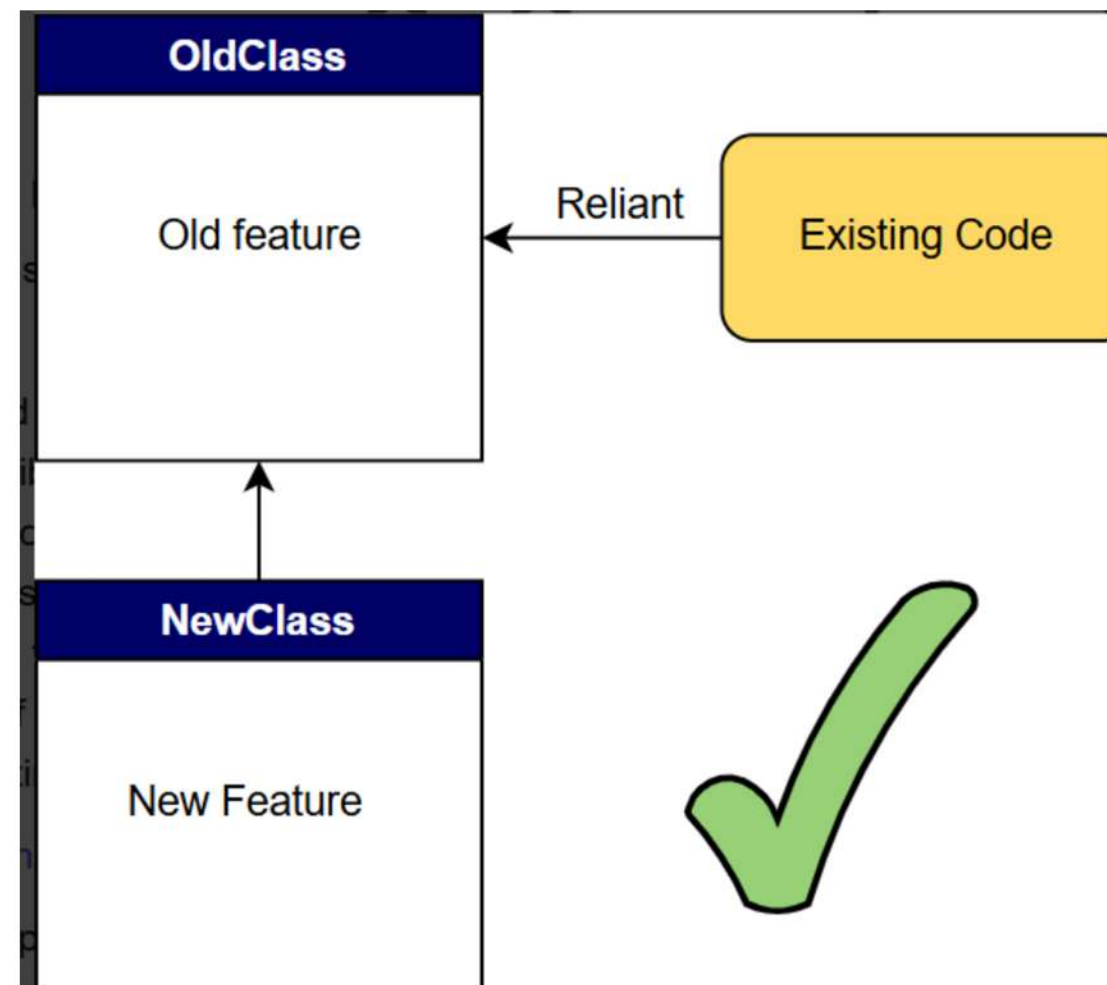
S: Single-responsibility principle_#



SRP: Splitting a non-SRP class

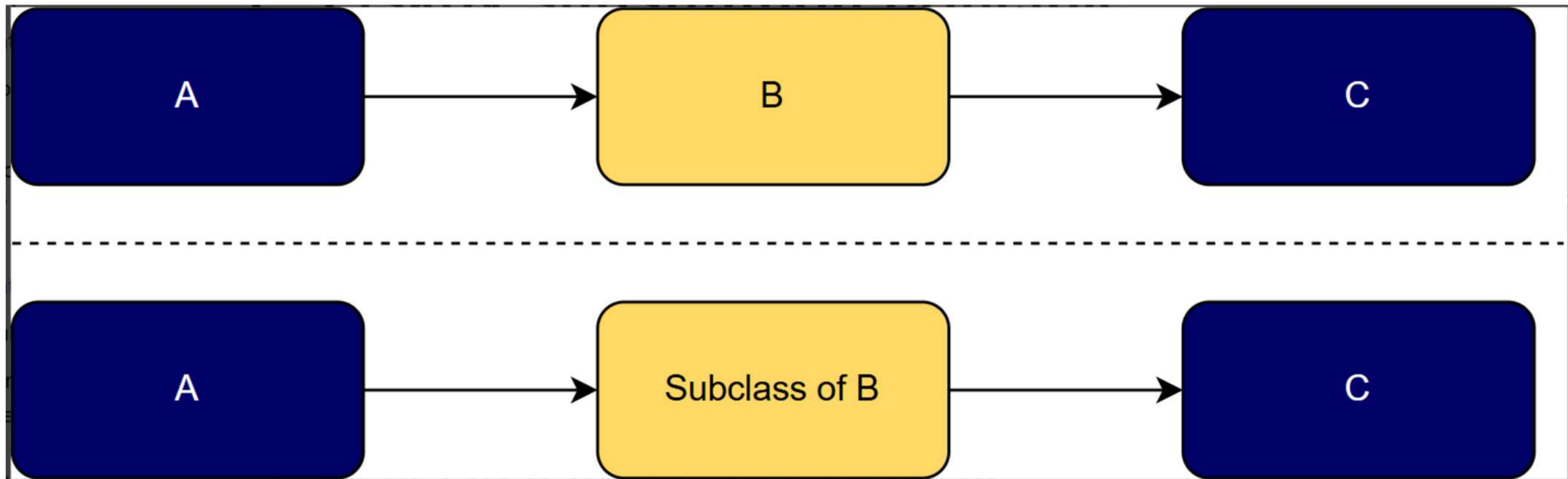
O5.2- OPEN CLOSED PRINCIPLE(OCP)

- Software entities like classes, modules & functions should be open for extension but closed for modification.
- Closed => may be compiled, stored in a library, and used by client classes.
- Open => any new class may use it as parent, adding new features.



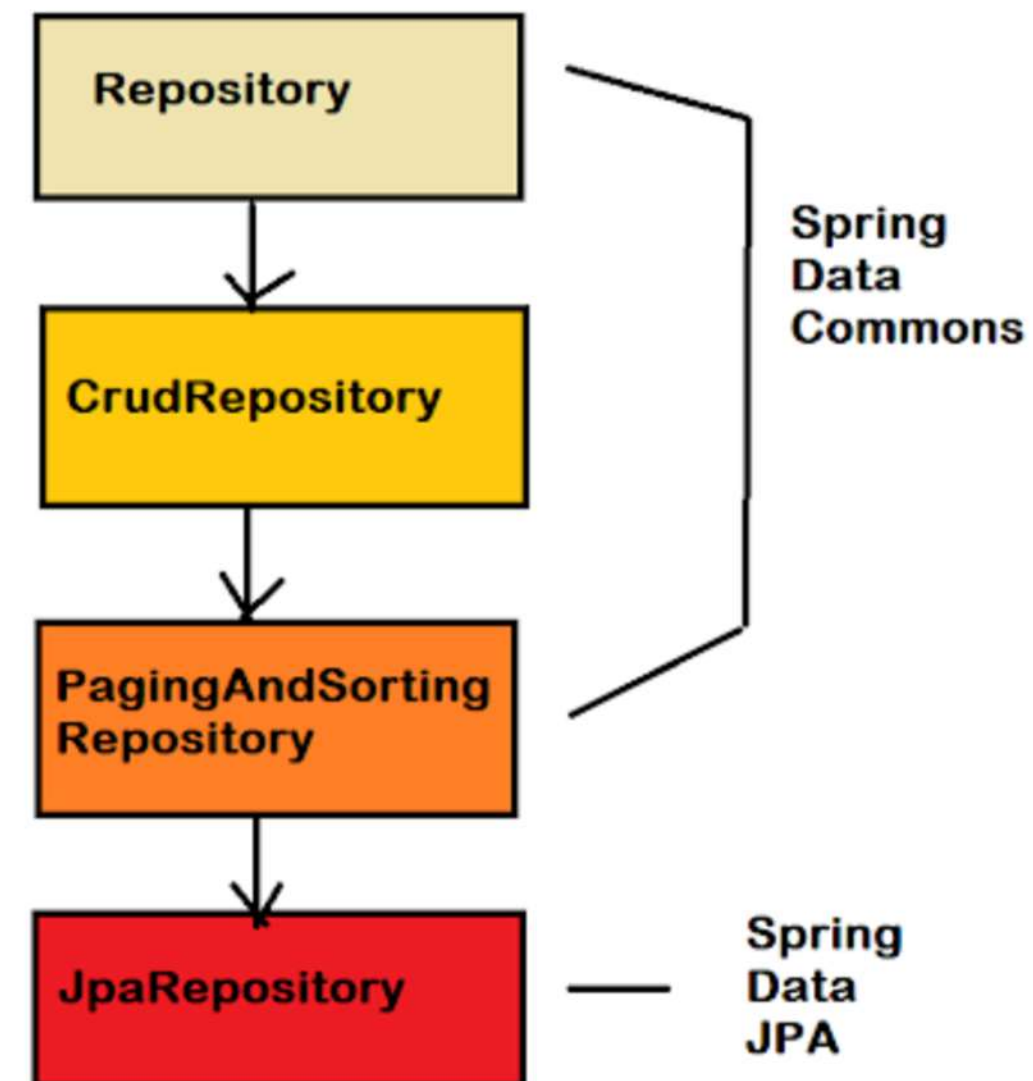
O5.3- LISKOV SUBSTITUTION PRINCIPLE (LSP)

- Derived types must be completely substitutable for their base types.
- If class A is a subtype of class B, then we should be able to replace B with A without disrupting the behavior of our program.



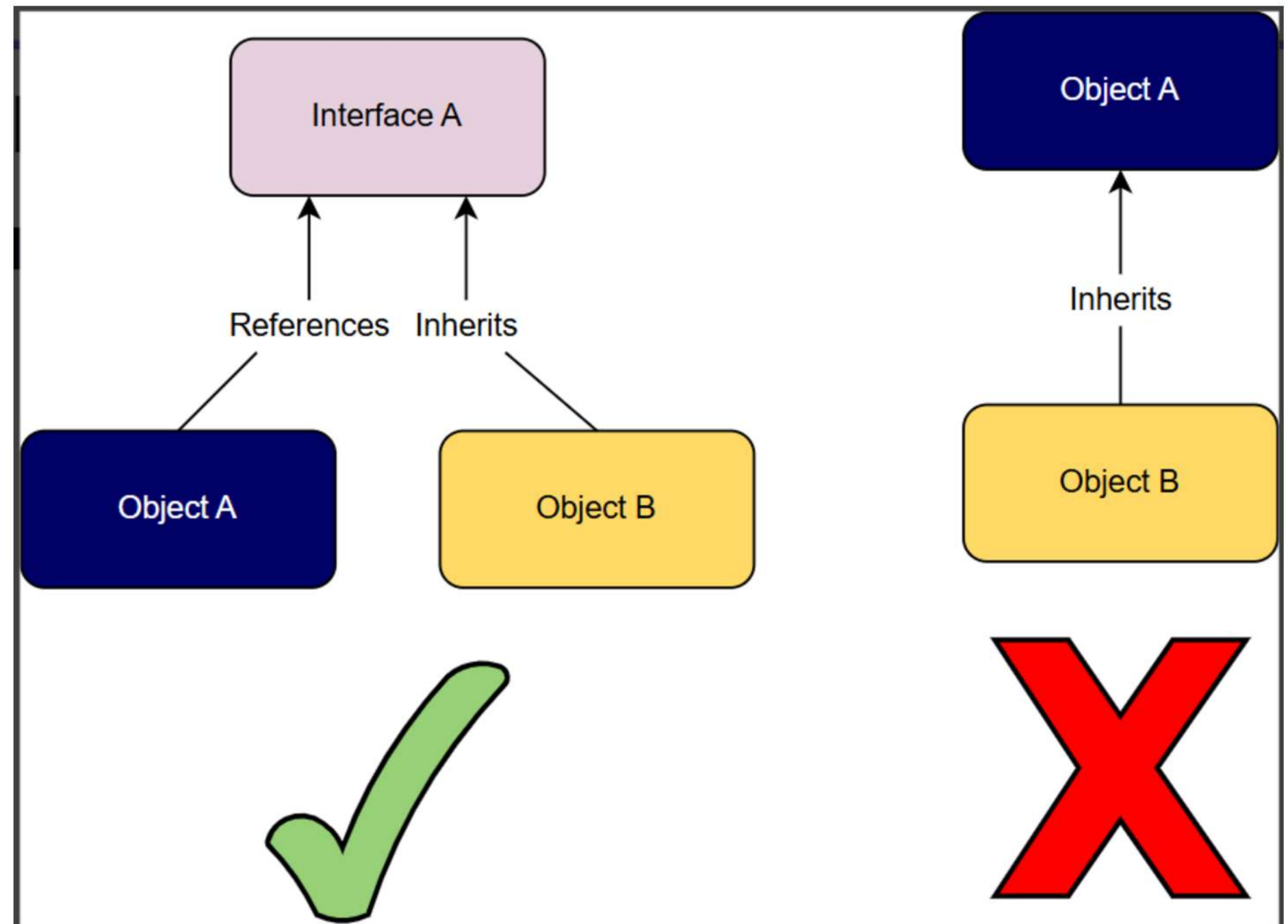
O5.4- INTERFACE SEGREGATION PRINCIPLE (ISP)

- Clients should not be forced to depend upon interface that they don't use
- Goal: Reduce the side effects of using larger interfaces by breaking application interfaces into smaller ones.



O5.5- **D**EPENDENCY INVERSION PRINCIPLE (DIP)

- Abstractions should not depend on details, Details should depend on abstraction.
- High-Level modules should not depend on low level modules, Both should depend on abstractions.



The background features a light cream color with four abstract teal shapes, one in each corner. Each teal shape is outlined with a thin gold line. Scattered around these shapes are several small teal dots of varying sizes.

THANK YOU